VEHICLE RENTAL MANAGEMENT SYSTEM

Project Report – Application Modules, Database Design and SQL Implementation

Python Flask • HTML • CSS • JavaScript • MySQL

Project	Vehicle Rental Management System
Database	MySQL – vehicle_rental_db
Application flow	Login → vehicles/customers → rental → payment → maintenance → reports
Database objects	7 tables, 2 views, 2 stored procedures, 3 triggers
Report note	Student will insert actual output screenshots and ER diagram.

Prepared for DBMS Mini Project Submission

1. Project Overview

The Vehicle Rental Management System is a database-driven web application designed to organize vehicle rental operations. It maintains vehicle and customer records, rentals, payments, maintenance history, and reports in a centralized MySQL database. The web interface is built with HTML, CSS, and JavaScript; the backend uses Python Flask, and database access uses mysql-connector-python.

Main Modules

Module	Purpose
Dashboard	Provides a summary view and navigation to application modules.
Vehicle Management	Adds, displays, edits, and deletes vehicle records.
Customer Management	Maintains customer contact and driving-licence details.
Rental Management	Stores customer, vehicle, rental dates, amount, and rental status.
Payment Management	Records payment date, amount, method, status, and transaction reference.
Maintenance Management	Records vehicle service date, description, cost, status, and next service date.
Reports	Displays rental details, revenue by month, and vehicle usage.

2. Application Dashboard

The dashboard is the main landing page after administrator login and provides access to the system's management modules.

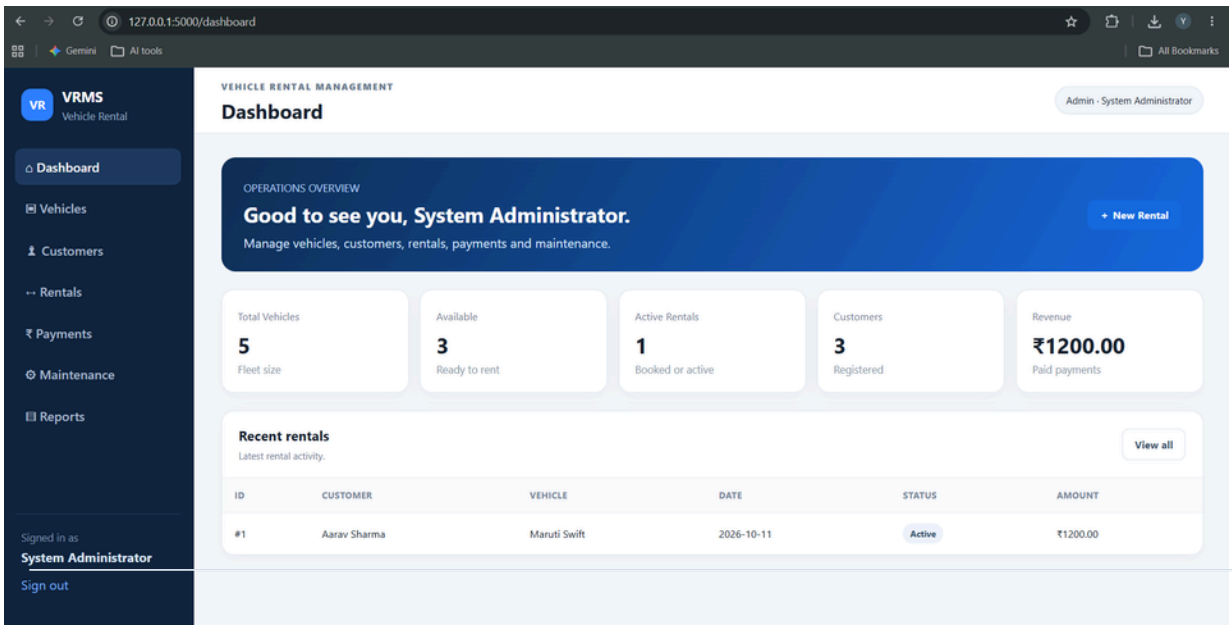


Figure1.Dashboard of the Vehicle Rental Management System.

3. Vehicle Management

The Vehicle Management module maintains the fleet. Vehicle records include category, registration number, brand, model, manufacture year, colour, fuel type, transmission, seating capacity, and current status. The module was tested by adding, editing, listing, and deleting a test vehicle.

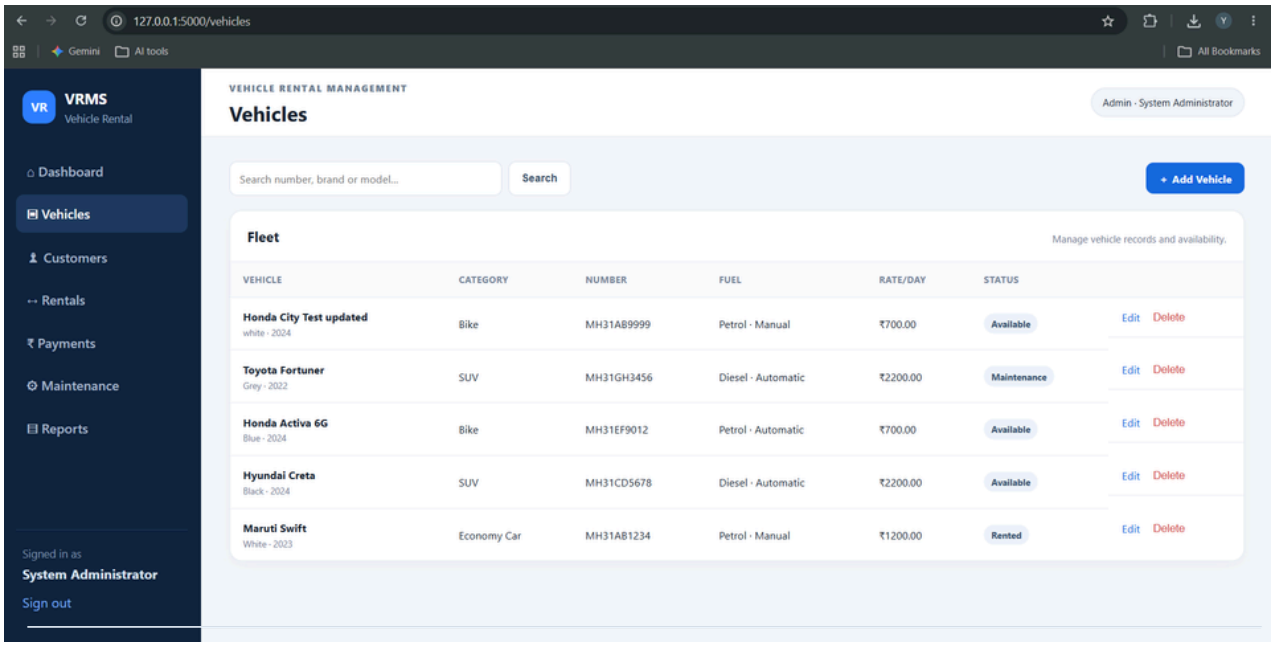


Figure 2. Vehicle management interface.

4. Customer Management

The Customer Management module stores customer information required for rental transactions, such as name, email, phone, address, and driving-licence number. Customers are associated with rental records through customer_id.

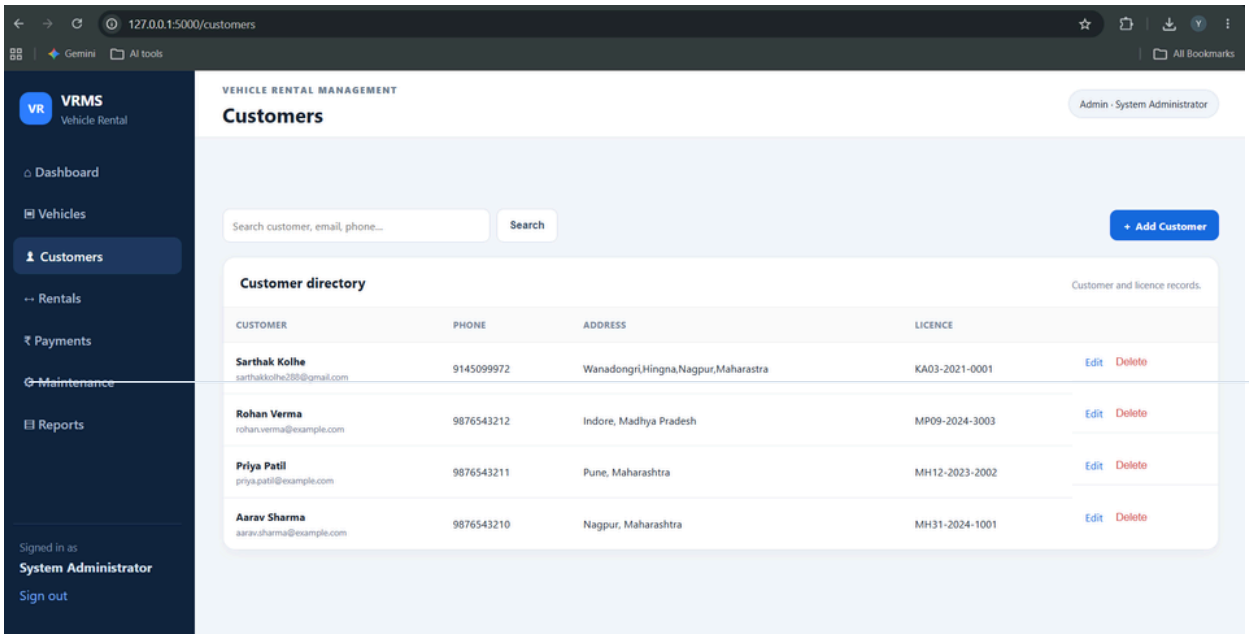


Figure 3. Customer management interface.

5. Rental Management

The Rental Management module links a customer with a vehicle for a rental period. A rental record stores rental date, expected return date, actual return date when available, daily rate, total amount, status, notes, and creation time. A test rental was created and verified in the application and reports.

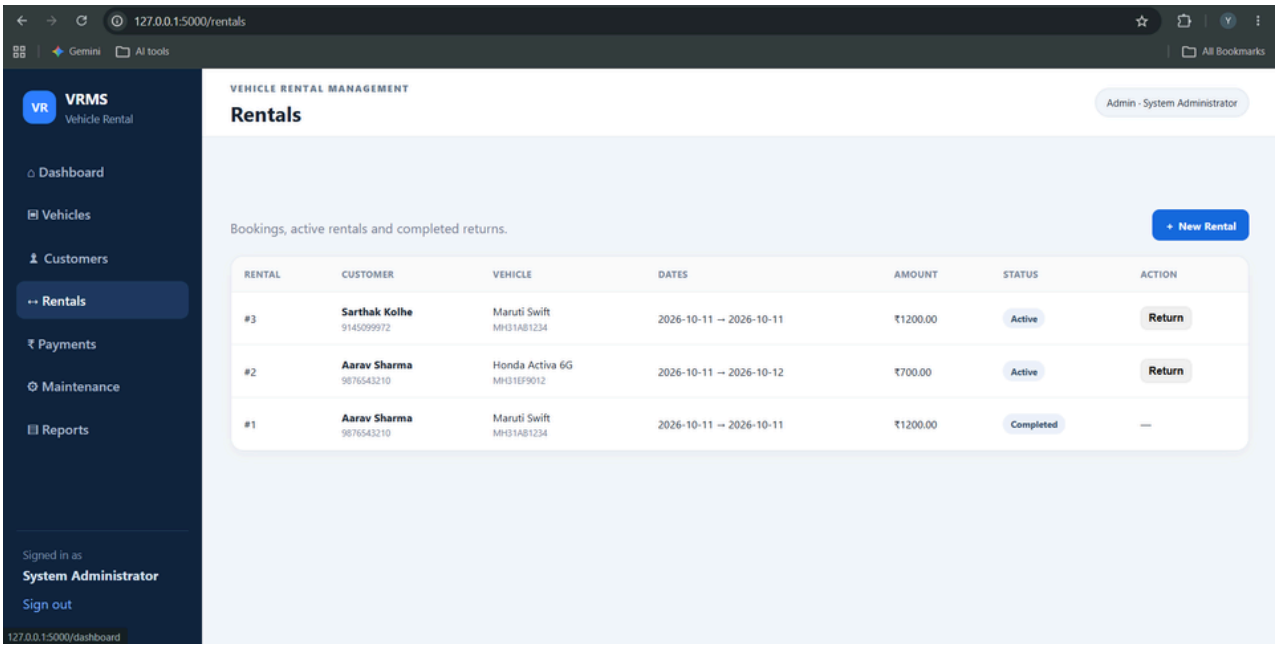


Figure 4. Rental records in the application.

6. Payment Management

The Payment Management module stores payments associated with rentals. Payment information includes rental identifier, payment date, amount, payment method, payment status, and transaction reference. A test payment was saved and displayed in the payment list.

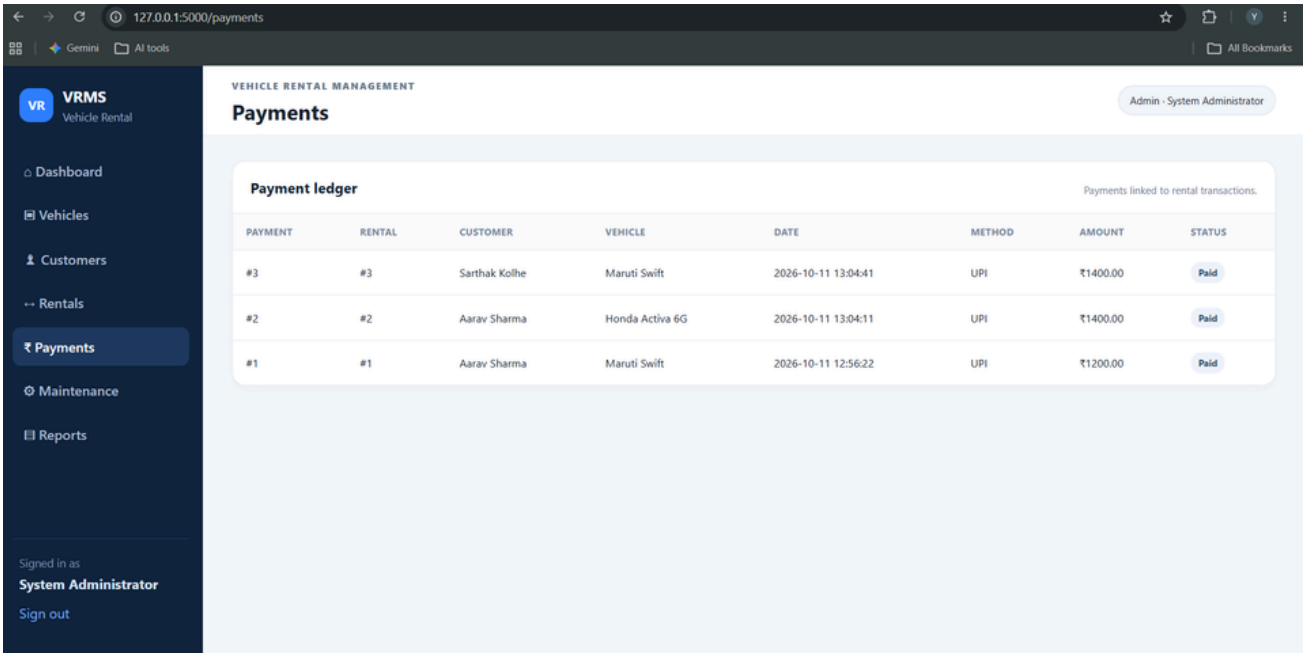


Figure 5. Payment management interface.

7. Vehicle Maintenance

The Maintenance module records servicing and repair activity for vehicles. Each maintenance record is associated with a vehicle and can include service date, description, cost, status, and next service date. It provides a service history and helps track maintenance expenditure.

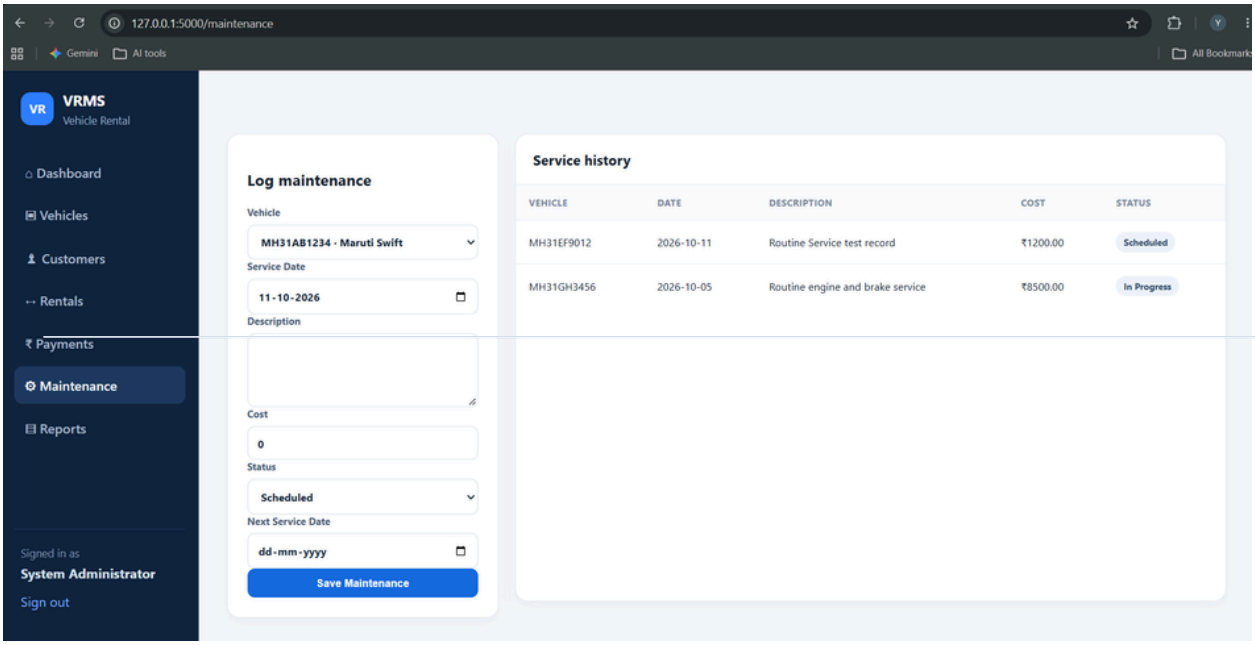


Figure 6. Vehicle service history and maintenance form.

8. Reports

The Reports page presents data derived from the database. The tested page showed rental details, revenue by month, and a vehicle usage section. During testing, the October 2026 revenue summary displayed three payments and ■4,000 revenue; this reflects the sample data present at that time and can change when records change.

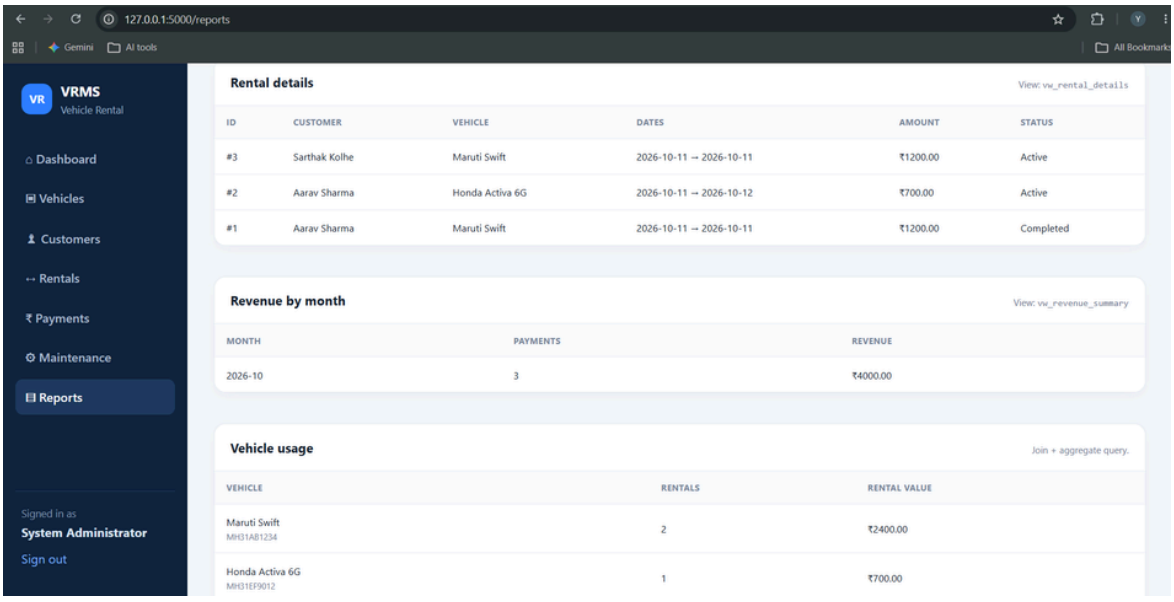


Figure 7. Reports showing rental details and revenue summary.

9. Entity Relationship Diagram

The core entities are users, customers, vehicle_categories, vehicles, rentals, payments, and maintenance. A category can contain multiple vehicles. A customer can have multiple rentals. A vehicle can be rented multiple times over time and can have multiple maintenance records. Rentals connect customers and vehicles; payments are associated with rentals.

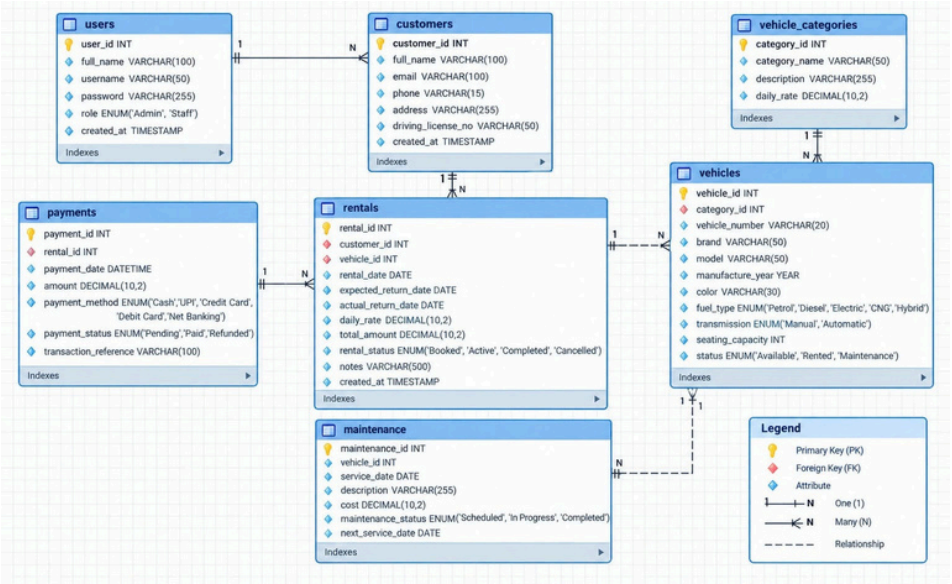


Figure 8. Entity Relationship Diagram for the Vehicle Rental Management System.

Replace the placeholder with the final ER diagram exported from draw.io or MySQL Workbench. Make sure primary keys, foreign keys, and relationship lines are readable.

10. Database Structure

The database is named vehicle_rental_db. The schema contains seven tables and two views. Primary keys uniquely identify records, while foreign keys connect related entities. Confirm exact column definitions against the SQL script supplied with the project before submission.

Table	Purpose / relationship
users	Stores application user and role/login information.
customers	Stores customer details; referenced by rentals.
vehicle_categories	Stores vehicle category and daily-rate information.
vehicles	Stores fleet details and category relationship; referenced by rentals and maintenance.
rentals	Connects customers and vehicles and stores rental period, amount, and status.
payments	Stores payment records associated with rentals.
maintenance	Stores service history associated with vehicles.

Views

View	Purpose
vw_rental_details	Presents rental details with related customer and vehicle information.
vw_revenue_summary	Summarizes payment/revenue information for reporting.

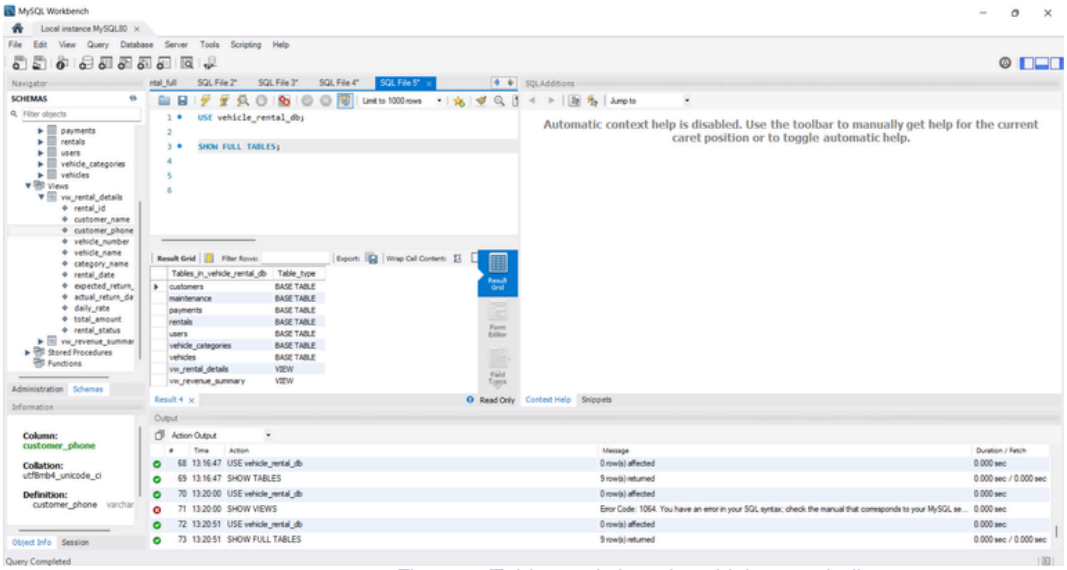
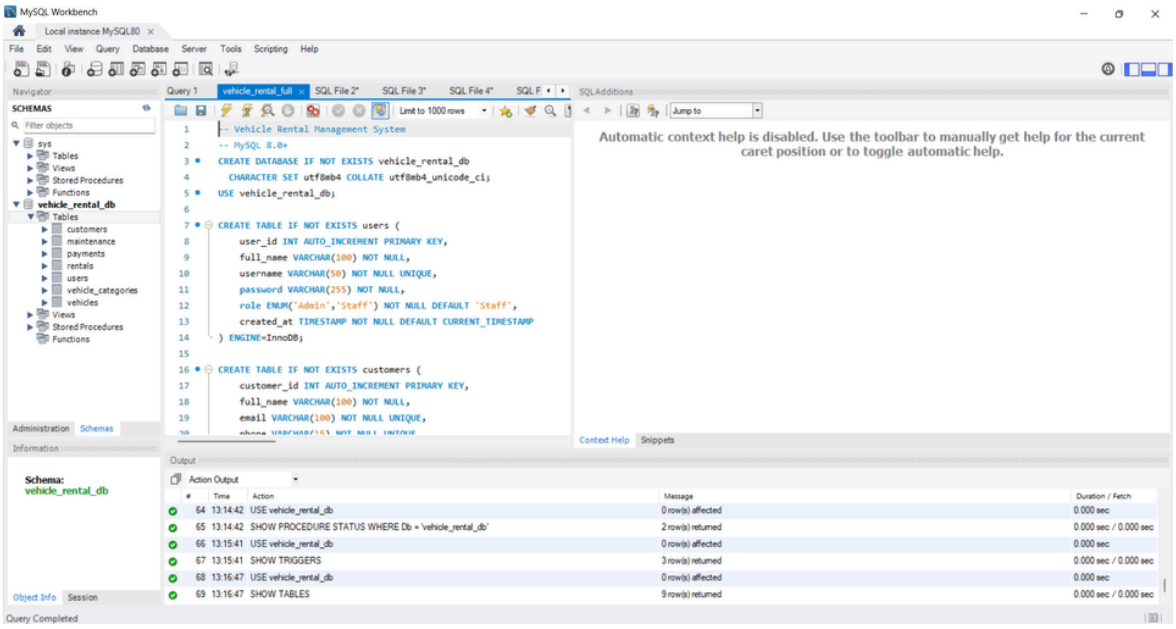
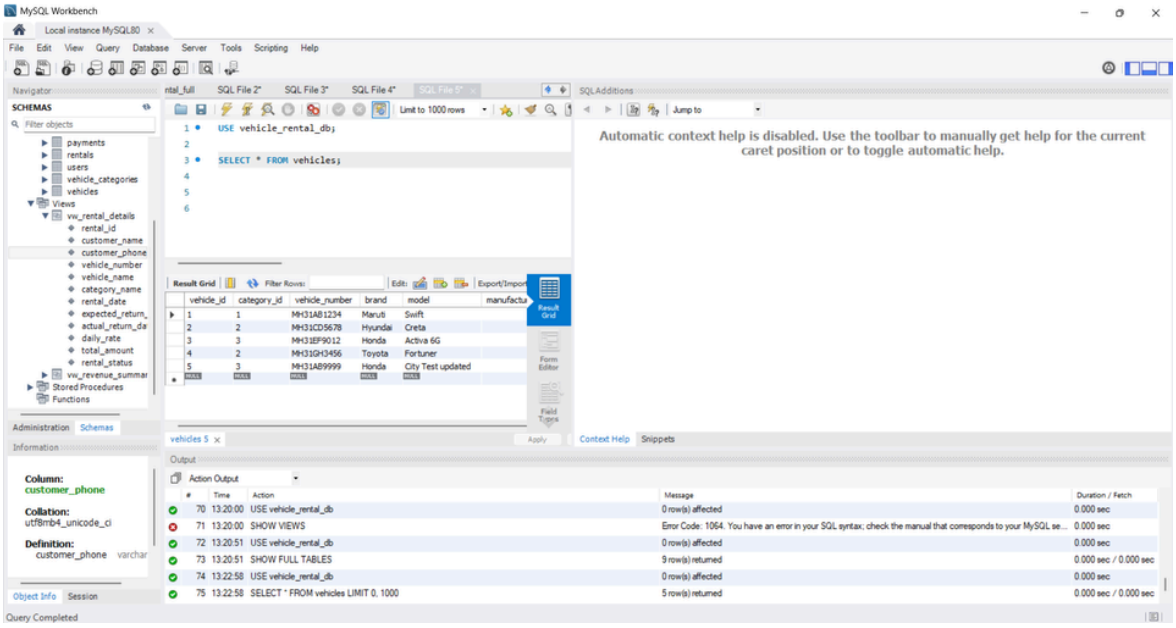


Figure 9. Tables and views in vehicle_rental_db.

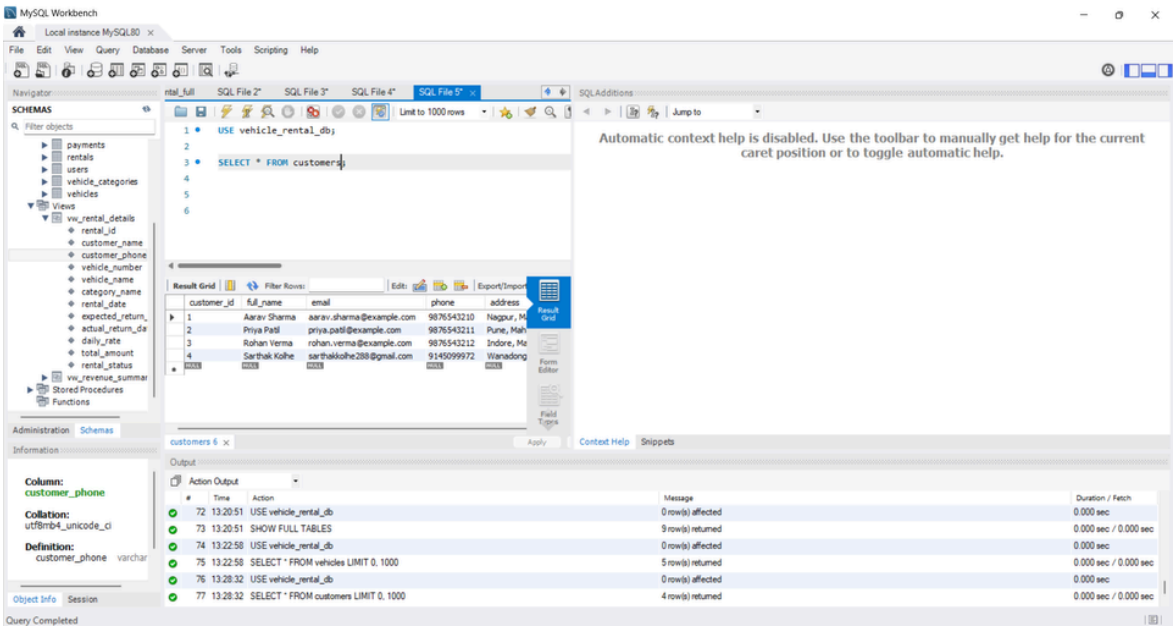
TABLES



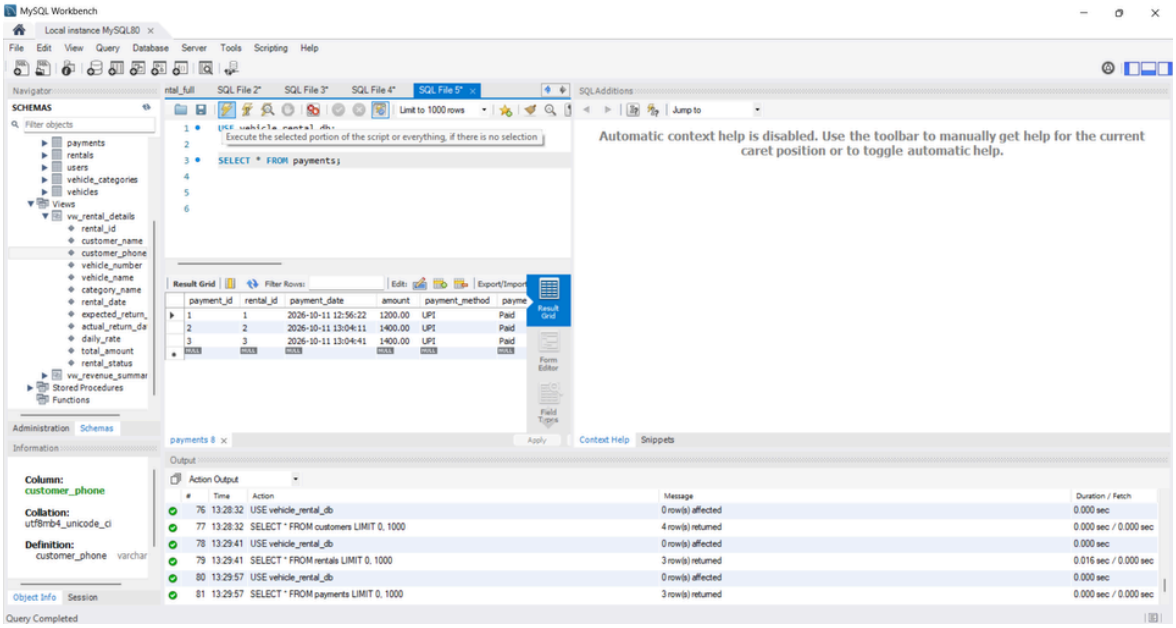
VEHICLES



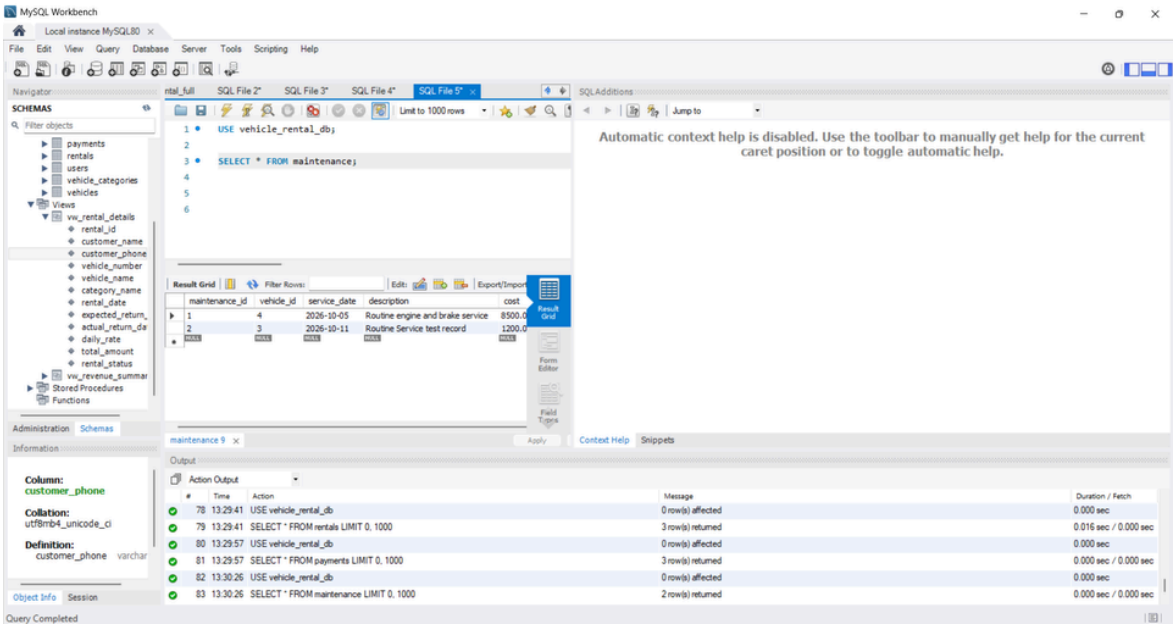
CUSTOMERS



PAYMENTS



MAINTENANCE



11. SQL Queries – Database Inspection

```
CREATE DATABASE IF NOT EXISTS vehicle_rental_db
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE vehicle_rental_db;
CREATE TABLE IF NOT EXISTS users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role ENUM('Admin','Staff') NOT NULL DEFAULT 'Staff',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(15) NOT NULL UNIQUE,
    address VARCHAR(255),
    driving_license_no VARCHAR(50) NOT NULL UNIQUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS vehicle_categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL UNIQUE,
    description VARCHAR(255),
    daily_rate DECIMAL(10,2) NOT NULL,
    CONSTRAINT chk_daily_rate CHECK (daily_rate > 0)
) ENGINE=InnoDB;
CREATE TABLE IF NOT EXISTS vehicles (
    vehicle_id INT AUTO_INCREMENT PRIMARY KEY,
    category_id INT NOT NULL,
    vehicle_number VARCHAR(20) NOT NULL UNIQUE,
    brand VARCHAR(50) NOT NULL,
    model VARCHAR(50) NOT NULL,
    manufacture_year YEAR,
    color VARCHAR(30),
    fuel_type ENUM('Petrol','Diesel','Electric','CNG','Hybrid') NOT NULL,
    transmission ENUM('Manual','Automatic') NOT NULL,
    seating_capacity INT NOT NULL,
    status ENUM('Available','Rented','Maintenance') NOT NULL DEFAULT 'Available',
    CONSTRAINT fk_vehicle_category FOREIGN KEY (category_id)
      REFERENCES vehicle_categories(category_id)
      ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_seating_capacity CHECK (seating_capacity > 0)
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS rentals (
  rental_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT NOT NULL,
  vehicle_id INT NOT NULL,
  rental_date DATE NOT NULL,
  expected_return_date DATE NOT NULL,
  actual_return_date DATE NULL,
  daily_rate DECIMAL(10,2) NOT NULL,
  total_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  rental_status ENUM('Booked','Active','Completed','Cancelled') NOT NULL
DEFAULT 'Booked',
  notes VARCHAR(500),
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_rental_customer FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_rental_vehicle FOREIGN KEY (vehicle_id)
    REFERENCES vehicles(vehicle_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_rental_dates CHECK (expected_return_date >=
rental_date),
  CONSTRAINT chk_rental_rate CHECK (daily_rate > 0),
  CONSTRAINT chk_total_amount CHECK (total_amount >= 0),
  CONSTRAINT chk_actual_return CHECK (actual_return_date IS NULL OR
actual_return_date >= rental_date)
) ENGINE=InnoDB;
CREATE TABLE IF NOT EXISTS payments (
  payment_id INT AUTO_INCREMENT PRIMARY KEY,
  rental_id INT NOT NULL,
  payment_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  amount DECIMAL(10,2) NOT NULL,
  payment_method ENUM('Cash','UPI','Credit Card','Debit Card','Net Banking')
NOT NULL,
  payment_status ENUM('Pending','Paid','Refunded') NOT NULL DEFAULT
'Pending',
  transaction_reference VARCHAR(100) UNIQUE,
  CONSTRAINT fk_payment_rental FOREIGN KEY (rental_id)
    REFERENCES rentals(rental_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_payment_amount CHECK (amount > 0)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS maintenance (
  maintenance_id INT AUTO_INCREMENT PRIMARY KEY,
  vehicle_id INT NOT NULL,
  service_date DATE NOT NULL,
  description VARCHAR(255) NOT NULL,
  cost DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  maintenance_status ENUM('Scheduled','In Progress','Completed') NOT NULL
DEFAULT 'Scheduled',
  next_service_date DATE,
  CONSTRAINT fk_maintenance_vehicle FOREIGN KEY (vehicle_id)
    REFERENCES vehicles(vehicle_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_maintenance_cost CHECK (cost >= 0)
) ENGINE=InnoDB;
```

```
CREATE INDEX idx_customer_name ON customers(full_name);
CREATE INDEX idx_vehicle_status ON vehicles(status);
CREATE INDEX idx_vehicle_category ON vehicles(category_id);
CREATE INDEX idx_rental_customer ON rentals(customer_id);
CREATE INDEX idx_rental_vehicle ON rentals(vehicle_id);
CREATE INDEX idx_rental_status ON rentals(rental_status);
CREATE INDEX idx_rental_date ON rentals(rental_date);
CREATE INDEX idx_payment_rental ON payments(rental_id);
CREATE INDEX idx_maintenance_vehicle ON maintenance(vehicle_id);


USE vehicle_rental_db;

INSERT INTO users (full_name, username, password, role)
SELECT 'System Administrator','admin','admin123','Admin'
WHERE NOT EXISTS (SELECT 1 FROM users WHERE
username='admin');

INSERT INTO users (full_name, username, password, role)
SELECT 'Rental Staff','staff','staff123','Staff'
WHERE NOT EXISTS (SELECT 1 FROM users WHERE username='staff');

INSERT INTO vehicle_categories (category_name, description,
daily_rate)
SELECT 'Economy Car','Affordable cars for city travel',1200.00
WHERE NOT EXISTS (SELECT 1 FROM vehicle_categories WHERE
category_name='Economy Car');

INSERT INTO vehicle_categories (category_name, description,
daily_rate)
SELECT 'SUV','Spacious sport utility vehicles',2200.00
WHERE NOT EXISTS (SELECT 1 FROM vehicle_categories WHERE
category_name='SUV');

INSERT INTO vehicle_categories (category_name, description,
daily_rate)
SELECT 'Bike','Two-wheelers for convenient travel',700.00
WHERE NOT EXISTS (SELECT 1 FROM vehicle_categories WHERE
category_name='Bike');

INSERT INTO vehicle_categories (category_name, description,
daily_rate)
SELECT 'Luxury Car','Premium vehicles for special occasions',4500.00
WHERE NOT EXISTS (SELECT 1 FROM vehicle_categories WHERE
category_name='Luxury Car');

INSERT INTO customers
(full_name,email,phone,address,driving_license_no)
SELECT 'Aarav
Sharma','aarav.sharma@example.com','9876543210','Nagpur,
Maharashtra','MH31-2024-1001'
WHERE NOT EXISTS (SELECT 1 FROM customers WHERE
email='aarav.sharma@example.com');

INSERT INTO customers
(full_name,email,phone,address,driving_license_no)
SELECT 'Priya Patil','priya.patil@example.com','9876543211','Pune,
Maharashtra','MH12-2023-2002'
WHERE NOT EXISTS (SELECT 1 FROM customers WHERE
email='priya.patil@example.com');
m.vehicle_id=v.vehicle_id AND m.service_date='2026-10-05');
```

```
DROP VIEW IF EXISTS vw_rental_details;
CREATE VIEW vw_rental_details AS
SELECT
   r.rental_id,
   c.full_name AS customer_name,
   c.phone AS customer_phone,
   v.vehicle_number,
   CONCAT(v.brand, ' ', v.model) AS vehicle_name,
   vc.category_name,
   r.rental_date,
   r.expected_return_date,
   r.actual_return_date,
   r.daily_rate,
   r.total_amount,
   r.rental_status
FROM rentals r
JOIN customers c ON c.customer_id = r.customer_id
JOIN vehicles v ON v.vehicle_id = r.vehicle_id
JOIN vehicle_categories vc ON vc.category_id = v.category_id;

DROP VIEW IF EXISTS vw_revenue_summary;
CREATE VIEW vw_revenue_summary AS
SELECT
   DATE_FORMAT(p.payment_date,'%Y-%m') AS revenue_month,
   COUNT(p.payment_id) AS payment_count,
   SUM(CASE WHEN p.payment_status='Paid' THEN p.amount ELSE 0 END) AS
paid_revenue
FROM payments p
GROUP BY DATE_FORMAT(p.payment_date,'%Y-%m');


USE vehicle_rental_db;
DROP TRIGGER IF EXISTS trg_rental_before_insert;
DROP TRIGGER IF EXISTS trg_rental_after_insert;
DROP TRIGGER IF EXISTS trg_rental_after_update;
DELIMITER $$

CREATE TRIGGER trg_rental_before_insert
BEFORE INSERT ON rentals
FOR EACH ROW
BEGIN
   DECLARE v_status VARCHAR(20);
   DECLARE v_rate DECIMAL(10,2);
 SELECT v.status, vc.daily_rate
     INTO v_status, v_rate
   FROM vehicles v
   JOIN vehicle_categories vc ON vc.category_id = v.category_id
   WHERE v.vehicle_id = NEW.vehicle_id
   FOR UPDATE;
   IF v_status <> 'Available' THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Selected vehicle is not available.';
   END IF;
   SET NEW.daily_rate = v_rate;
   SET NEW.total_amount = GREATEST(DATEDIFF(NEW.expected_return_date,
NEW.rental_date), 1) * v_rate;
   SET NEW.rental_status = 'Active';
END$$
```

```
CREATE TRIGGER trg_rental_after_insert
AFTER INSERT ON rentals
FOR EACH ROW
BEGIN
   UPDATE vehicles
     SET status='Rented'
    WHERE vehicle_id=NEW.vehicle_id;
END$$

CREATE TRIGGER trg_rental_after_update
AFTER UPDATE ON rentals
FOR EACH ROW
BEGIN
   IF NEW.rental_status='Completed' OR NEW.rental_status='Cancelled' THEN
      UPDATE vehicles
        SET status='Available'
       WHERE vehicle_id=NEW.vehicle_id;
   END IF;
END$$

DELIMITER ;


USE vehicle_rental_db;

DROP PROCEDURE IF EXISTS sp_create_rental;
DROP PROCEDURE IF EXISTS sp_complete_rental;

DELIMITER $$

CREATE PROCEDURE sp_create_rental(
   IN p_customer_id INT,
   IN p_vehicle_id INT,
   IN p_rental_date DATE,
   IN p_expected_return_date DATE,
   IN p_notes VARCHAR(500),
   OUT p_rental_id INT
)
BEGIN
   INSERT INTO rentals (
      customer_id, vehicle_id, rental_date,
 expected_return_date, notes
 )
 VALUES (
 p_customer_id, p_vehicle_id, p_rental_date,
 p_expected_return_date, p_notes
 );

 SET p_rental_id = LAST_INSERT_ID();
END$$

CREATE PROCEDURE sp_complete_rental(
 IN p_rental_id INT,
 IN p_actual_return_date DATE
)
BEGIN
 DECLARE v_start DATE;
 DECLARE v_rate DECIMAL(10,2);
```

```
 SELECT rental_date, daily_rate
 INTO v_start, v_rate
 FROM rentals
 WHERE rental_id=p_rental_id
 FOR UPDATE;

 UPDATE rentals
 SET actual_return_date=p_actual_return_date,
 total_amount=GREATEST(DATEDIFF(p_actual_return_date, v_start),1) * v_rate,
 rental_status='Completed'
 WHERE rental_id=p_rental_id;
END$$

DELIMITER ;
```

12. Views, Stored Procedures and Triggers

Views

The database contains vw_rental_details and vw_revenue_summary. A view is a saved SQL query that presents related records in a convenient format and can simplify report queries.

Stored Procedures

The database contains sp_create_rental and sp_complete_rental. Stored procedures group database operations under named routines. Refer to the project's SQL file for their exact parameters and implementation.

Triggers

The database contains three rental-related triggers: trg_rental_before_insert, trg_rental_after_insert, and trg_rental_after_update. Triggers execute automatically in response to specified table events. Their exact effects should be explained using their definitions in the SQL script.

13. Database Integrity and Design Notes

The design separates customer, vehicle, category, rental, payment, maintenance, and user data into related tables rather than storing everything in one table. This supports clearer relationships and reduces unnecessary duplication.

- Primary keys provide unique identifiers for records.
- Foreign keys connect related entities where defined in the schema.
- Rentals connect customers and vehicles for a defined period.
- Payment details are stored separately from rental details.
- Maintenance records preserve service history independently of rental activity.
- Views support reporting; stored procedures and triggers support database-side operations.

14. Application Workflow and Testing

Application workflow

- The administrator signs in to the web application.
- Vehicle and customer information is maintained through the respective modules.
- A rental is created by selecting a customer and available vehicle and entering rental details.
- Payment details are saved and viewed in the Payments module.
- Maintenance activity is recorded against a vehicle.

• The Reports page displays rental details, revenue summaries, and vehicle usage information.

Functional testing performed

Test area	Observed result
Dashboard	Opened successfully.
Vehicles	Add, edit, list, and delete test operations completed.
Customers	A test customer was saved and displayed.
Rentals	A rental was created and displayed.
Payments	A payment was saved and displayed.
Maintenance	A maintenance record was saved and displayed.
Reports	Rental details and monthly revenue were displayed.
Views	vw_rental_details and vw_revenue_summary were present.
Stored procedures	sp_create_rental and sp_complete_rental were present.
Triggers	Three rental-related triggers were present.
Restart test	Flask restarted and login opened successfully.

15. Conclusion

The Vehicle Rental Management System demonstrates how a relational database can support vehicle rental operations. It combines web-based record management with MySQL tables, views, stored procedures, triggers, and report queries. Testing confirmed that the main application modules opened and that test records could be saved and viewed. The project demonstrates database design, application-to-database connectivity, and CRUD operations.

Before submission, insert the actual output screenshots and final ER diagram, check the report against the final SQL script, and add the public GitHub repository URL.

16. Repository and Project Details

Item	Details
Project	Vehicle Rental Management System
Project type	DBMS Mini Project
Frontend	HTML, CSS, JavaScript
Backend	Python Flask
Database	MySQL — vehicle_rental_db
Database connector	mysql-connector-python
Version control	Git and GitHub
Repository URL	Add your GitHub repository URL before submission.

GitHub Repository

https://github.com/Yash9145/Vehicle-Rental-Management-System.git